SIMATS ENGINEERING

Department of Artificial Intelligence & Machine Learning

ITA05 – Computer Vision

ASSIGNMENT 1 – CO1

Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Vision-Based Mango Grading Line

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

STUDENT DETAILS

Particulars	**Details**	**Particulars**	**Details**
Student Name	M. Sahasra	Register Number	192425317
Department / Section	Artificial Intelligece & Machine Learning	Academic Year / Semester	2025-2026
Faculty Name	Dr. Bindu G	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	_____	Course Code	ITA05

Assignment Title: Apply, Compare and Justify Image Acquisition, Formation and Processing Techniques for a Vision-Based Mango Grading Line (Total Marks – 50)

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Grading approaches and computer vision features	4		
2	Levels of computer vision and pipeline depth	5		
3	Image representations for ripeness	4		
4	Pixel connectivity for defect counting	4		
5	Image sensing and acquisition	5		
6	Spatial sampling and resolution	5		
7	Intensity quantization	4		
8	Image formation, optics and lighting	5		
9	Integrated implementation and performance	6		

10	Adaptation to revised inspection requirements	8		
TOTAL		50		

OVERALL REMARKS

Student Signature	Faculty Signature	Program Director

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present ✗ = missing / not satisfied P = partly present. Based on the "Evidence Expected" of each task. Figures and screen outputs are referenced in the report but are not present in the submitted text copy.

Task	Evidence expected	Status	Notes
1	Agreement/timing table, feature comparison and justified grading approach		
2	Level-annotated pipeline, accuracy/time comparison and selected pipeline		
3	Channel plots, accuracy table, array view and storage calculation		
4	Labelled defect outputs, count comparison and distance-measure table		
5	Acquisition simulations, blur calculation and sensor comparison		
6	Resolution table, down-sampled		

	images, data rate and storage calculations		
7	Re-quantized images, bruise contrast, detection and storage comparison		
8	Focal-length and perspective calculations, illumination maps and correction results		
9	Configuration comparison, integrated architecture and decision table		
10	Revised requirements, RGB/NIR comparison and final redesign justification		

IMPLEMENTATION EVIDENCE CHAIN (Tasks 1–10)

Stage	Evidence to retain	Used in tasks
Image acquisition	Original images, exposure settings, sensor and lighting details	1, 5, 8, 9, 10
Image formation	Calibration images, lens parameters, flat-field reference	5, 8, 9, 10
Sampling	Original and resized images, mm/pixel calculation	6, 9, 10
Quantization	8/6/5/4-bit images and intensity histograms	7, 9, 10
Representation	RGB, HSV and Lab channels; class scatter plots	3, 9, 10
Segmentation	Binary masks and connected-component labels	2, 4, 9, 10
Feature extraction	Fruit dimensions, spot count, colour and bruise features	1, 2, 4, 9, 10
Classification	Predicted grade, confidence and reference label	1, 2, 9, 10
Decision and control	Accept/reject/review output and ejector timing	1, 9, 10

SUBMISSION CHECKLIST (not separately marked)

- ☐ All ten tasks are answered in the given order.
- ☐ Each task compares all three specified techniques.
- ☐ Every numerical result is labelled as measured, calculated, estimated or illustrative.
- ☐ Python/OpenCV source code is commented and reproducible.
- ☐ Tables include units and clearly stated assumptions.
- ☐ Figures and output screenshots are numbered and captioned.
- ☐ Simulated results are not described as real-world measurements.
- ☐ The final decision in each task is supported by comparison evidence.
- ☐ Original image evidence is retained for audit and verification.

COMPUTER VISION

ASSIGNMENT 1 – CO1

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Nature of Assignment: Individual

Duration: 1 Week

Total Marks: 50

Submission Date: _____

Scenario – Vision-Based Mango Grading Line

GreenHarvest Mango Processing Unit, a farmers' cooperative near Krishnagiri, grades export mangoes manually. Human graders may become tired and disagree with each other. The unit wants to introduce a computer vision system above its conveyor to measure fruit size, estimate ripeness, detect defects and automatically separate export-quality mangoes from defective fruit.

You are the vision engineer. In every task, three candidate techniques are compared. Each technique is explained, implemented or modelled where possible, evaluated against the stated requirements, and followed by a justified selection. Any assumptions and limitations are stated clearly.

Operating Data Used for All Tasks

Parameter	**Value**
Conveyor speed	0.5 m/s (Task 10 changes this to 1.0 m/s)
Field of view	400 mm across the belt; 300 mm along the conveyor
Mango length	70–110 mm
Maximum fruit height	90 mm
Throughput	8 fruits/second (Task 10: 16 fruits/second)
Decision time	1.2 seconds (Task 10: 0.6 seconds)
Smallest original defect	2 mm (Task 10: 0.5 mm)

Bruise contrast	Approximately 6–10 grey levels on an 8-bit image
Camera working distance	500 mm
Processing hardware	4-core CPU, 16 GB RAM, no dedicated GPU
Storage	2 TB
Imaging budget	₹1,50,000
Retention period	7 days

Experimental Setup Used for All Tasks

A consistent experimental setup is required so that comparisons across the ten tasks use the same assumptions. The reference answer document reports several results from controlled or generated images. Those values are identified as reference experimental results; they must not be represented as measurements newly obtained on another computer.

Item	**Setup / assumption**
Image dataset	At least 30 mango images for grading comparisons; larger task-specific sets where stated.
Software	Python, OpenCV and NumPy; scikit-learn may be used for KNN/SVM where needed.
Image format	OpenCV BGR input; 8-bit image processing unless the task specifies otherwise.
Ground truth	Defect labels, defect dimensions and ripeness labels recorded for each test image.
Timing	Use time.perf_counter(); report median and/or mean with the test hardware stated.
Reproducibility	Use fixed random seeds for generated/simulated experiments.

Evidence status	Separate actual captured images, synthetic images, calculated values and theoretical discussion.

Why controlled or generated images may be used. When real mango photographs with measured defect dimensions are unavailable, synthetic images can provide known ground truth for testing individual factors. However, synthetic results do not replace validation on real fruit because they may not reproduce natural texture, wax reflections, dirt, fruit overlap or all forms of illumination variation.

Why some advanced models may not be executed. Vision–language models, pretrained embeddings, segmentation networks and depth-estimation models may require model downloads, compatible libraries, substantial memory or GPU support. If they are not actually run, their role and limitations should be described theoretically without inventing accuracy or latency measurements.

Shared Processing Workflow

The common rule-based workflow consists of image loading, denoising, colour conversion, fruit segmentation, feature extraction, defect analysis and a final grading decision. The same core workflow can be reused in Tasks 1, 2 and 5–10, with task-specific changes to calibration, sampling, quantization and sensing.

```
import cv2
import numpy as np

def preprocess(bgr):
    # Smooth small sensor noise and convert to HSV
    blurred = cv2.GaussianBlur(bgr, (5, 5), 0)
    hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
    return blurred, hsv

def segment_fruit(hsv):
    # Example broad mask; thresholds must be calibrated
```

```
    lower = np.array([5, 35, 30], dtype=np.uint8)
    upper = np.array([35, 255, 255], dtype=np.uint8)
    mask = cv2.inRange(hsv, lower, upper)
    kernel = np.ones((5, 5), np.uint8)
    mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
    return mask


def grade_from_features(spot_count, max_spot_mm, streak, bruise):
    # Buyer rule: reject if any specified defect is present
    reject = (spot_count > 3 or max_spot_mm > 4.0 or streak or bruise)
    return 'REJECT' if reject else 'EXPORT'
```

The thresholds above are starting examples, not universal mango thresholds. They must be calibrated using representative images and verified against manually labelled ground truth.

TASK 1 – Overview and Features of Computer Vision: Compare Grading Approaches [4 Marks]

The cooperative must decide whether manual grading, image-processing-assisted inspection or automated computer vision is most suitable for the production line. The buyer rejects mangoes with more than three black spots, any spot larger than 4 mm, a sap-burn streak or a bruise.

QUESTION AND REQUIREMENTS

Before buying any hardware, the cooperative asks whether a computer vision system is the right answer at all. Compare three ways of grading the fruit.

Implement three mango quality-grading approaches using Python/OpenCV and a dataset of at least 30 mango images: (A) manual grading by three evaluators across two rounds, (B) image-processing-assisted visual inspection, and (C) rule-based automated grading. Develop the commented source code, record grading decisions, and obtain experimental results through actual execution and measurements.

Analyze the performance of the three grading approaches with respect to processing speed (fruits/s), repeatability, objectivity, non-contact operation, measurement precision (mm), continuous operation, traceability, adaptability to new defect types, and implementation and maintenance cost.

Critique the suitability of the three approaches for an industrial throughput requirement of 8 fruits per second by measuring processing time and throughput and calculating inter-rater and intra-rater agreement using appropriate statistical measures, such as Cohen's kappa.

Differentiate image processing, computer vision, machine vision, and computer graphics by establishing their respective input–output transformations and demonstrating their roles in the mango quality-inspection workflow.

Evaluate the production capacity and operational limitations of manual grading at 8 fruits per second over a 10-hour shift, and assess the consequences of an unrecognised defect type in a rule-based inspection system. Recommend suitable corrective measures based on the analysis.

Validate the performance of a vision–language model, such as Florence-2 or LLaVA, on 10 mango images using a predefined text prompt. Compare its grading decisions with those of the manual, image-processing-assisted, and rule-based approaches, and assess its suitability for

export-quality inspection based on repeatability, explainability, inference latency, and defect-detection rety.liabili

Evidence Expected: An agreement/timing table for the three approaches, a feature-by-feature comparison, and a justified choice with its limits stated.

Technique	Basic idea and key concepts / functions
A. Manual grading by trained workers	Human visual inspection under shed lighting; grader decides size, ripeness and defects by eye.
B. Image-processing aid	Camera image is enhanced (cv2.equalizeHist, cv2.resize, zoom) and shown on a monitor; the worker still makes every decision.
C. Automated computer vision system	Camera + processing extract measurements and interpret them (size, colour class, defect present) and trigger the ejector without human input.

Ans:

Three approaches are compared using the same mango set: manual grading by three evaluators over two rounds, image-processing-assisted visual inspection, and rule-based automated grading. The comparison considers accuracy, agreement, throughput, repeatability, traceability, measurement precision and cost.

1. Implementation of the Three Approaches

Approach	Implementation	Main output
A. Manual grading	Three trained evaluators inspect each fruit twice and record accept/reject decisions.	Human grading decision
B. Image-processing aid	A camera captures the fruit; brightness/contrast is enhanced and the image is	Human decision based on enhanced image

	enlarged for human inspection.	
C. Automated computer vision	OpenCV segments the fruit, measures dimensions and detects spots, streaks and bruises using rules.	Automatic accept/reject decision

For image enhancement, the V channel of HSV may be equalized and the image enlarged for inspection. For automated grading, Gaussian blur, HSV conversion, thresholding, contour detection and connected-component analysis are used.

```
def enhance_for_display(bgr):
    hsv = cv2.cvtColor(bgr, cv2.COLOR_BGR2HSV)
    hsv[:, :, 2] = cv2.equalizeHist(hsv[:, :, 2])
    enhanced = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
    return cv2.resize(enhanced, None, fx=2, fy=2,
                      interpolation=cv2.INTER_CUBIC)
```

2. Agreement and Timing Analysis

Cohen's kappa measures agreement beyond chance. For two sets of categorical decisions:

$\kappa = (Po - Pe) / (1 - Pe)$, where Po is observed agreement and Pe is expected agreement by chance.

Parameter	**Manual**	**Image-assisted**	**Automated**
Accuracy (reference experiment)	83.3%	90.0%	93.3%
Agreement (reported)	0.64	0.73	1.00
Throughput	Below 8 fruits/s	Limited by human review	18 fruits/s reported
Repeatability	Moderate	Good	High for fixed rules

The table reproduces the reported comparison from the reference answer. The reference human-grader decisions were simulated rather than collected from three real evaluators. Therefore, the agreement values demonstrate the experimental model and should be confirmed using actual grading sheets before an industrial claim is made.

3. Feature-by-Feature Comparison

Feature	Manual	Image-assisted	Automated
Speed	Low	Medium	High
Objectivity	Depends on worker	Partly objective	Rule-based and consistent
Non-contact	Yes	Yes	Yes
Measurement precision	Variable	Improved by zoom	Calibrated pixel measurement
Continuous operation	Difficult	Difficult	Suitable with maintenance
Traceability	Paper records	Saved images possible	Automatic logs and images
New defect adaptation	Training required	Training required	Rules/model must be updated
Initial cost	Low	Medium	Medium/high
Maintenance	Labour and training	Camera and display	Camera, lighting and software

4. Manual Grading Capacity

For a 10-hour shift: $10 \times 60 \times 60 = 36{,}000$ seconds. At 8 fruits per second, the line processes $36{,}000 \times 8 = 288{,}000$ fruits per shift. A single worker cannot inspect this volume at the required speed while maintaining consistent accuracy. Staffing requirements depend on the actual per-worker inspection rate and shift design.

A rule-based system may fail when a new defect type is not represented by its rules. Corrective measures include periodic review of rejected and accepted samples, adding validated rules,

collecting labelled examples, using a review category for uncertain cases and retraining a learned model when sufficient data is available.

5. Vision–Language Model Evaluation

A vision–language model such as Florence-2 or LLaVA can be prompted to describe ripeness and visible defects. A suitable prompt is: “Inspect this mango. Identify its ripeness, count visible spots, describe any streak or bruise, and classify it as export quality or defective according to the stated buyer rules.”

Criterion	**Rule-based vision**	**Vision–language model**
Repeatability	High for fixed inputs and thresholds	May vary with prompt/model settings
Explainability	Measurable features and rules	Natural-language explanation
Latency	Usually low	Model-dependent; may be higher
Unknown defects	Requires rule updates	May describe unfamiliar patterns, but can hallucinate
Audit suitability	Good with retained images and logs	Requires validation and controlled versioning

The reference discusses VLM evaluation but does not report an actual 10-image execution with measured latency and accuracy. No numerical VLM performance is therefore claimed here.

Final Decision: Automated computer vision is selected because it offers consistent decisions, measurable features, traceability and sufficient reported throughput. Human review should remain available for uncertain cases and newly observed defect types.

TASK 2 – Levels of Computer Vision and Image Processing: Compare Pipeline Depths [5 Marks]

Computer vision operations can be grouped into low-level, mid-level and high-level processing. The three candidate pipelines are low-level only, low plus mid-level, and low plus mid plus high-level vision.

QUESTION AND REQUIREMENTS

Simulate three computer vision techniques for fruit grading using Python/OpenCV and self-collected images. Submit commented source code and obtain all experimental results through code execution or actual measurements.

Analyze the three techniques in terms of grading accuracy, handling of touching fruits and leaf occlusion, robustness to illumination variations, explainability of grading decisions, processing time per fruit, and effort required to incorporate a new grade.

Assess the performance of all three techniques using the same labelled image dataset, including challenging cases. Tabulate the measured grading accuracy and processing time in milliseconds per image, and determine whether each technique satisfies the decision-time limit of 1.2 seconds.

Design and illustrate the selected computer vision pipeline by identifying each OpenCV processing stage as low-level, mid-level, or high-level vision and specifying its input and output representations, such as images, regions, feature vectors, and class labels.

Analyze the consequences of relying exclusively on low-level vision when dark stem ends or shadows are incorrectly identified as defects, and determine how inadequate mid-level features can affect the decisions made by the high-level grading stage.

Assess how end-to-end deep learning models, including Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs), learn low-level, mid-level, and high-level representations from labelled images. Compare their placement in the proposed pipeline and estimate their labelled-data requirements relative to the conventional computer vision approach.

Evidence Expected: A level-annotated pipeline diagram, an accuracy/time table for the three depths, and a justified choice of pipeline depth.

Technique	**Basic idea and key concepts / functions**
A. Low-level only	Pixel operations only – cv2.GaussianBlur, cv2.cvtColor, cv2.threshold – and a fixed rule

	on the count of dark pixels decides "defective".
B. Low + mid-level	Adds segmentation and feature extraction – cv2.findContours, cv2.contourArea, cv2.minAreaRect, mean hue, number of spots – and a rule-based grade.
C. Low + mid + high-level	Adds interpretation: features fed to a simple classifier (cv2.ml.KNearest or cv2.ml.SVM) that outputs the export grade with a confidence.

Ans:

1. Three Pipeline Techniques

Technique	**Operations**	**Output**
A. Low-level only	Gaussian blur, colour conversion, thresholding and pixel rules.	Processed image / binary mask
B. Low + mid-level	Adds segmentation, contours, dimensions, colour and defect features.	Regions and feature vector
C. Low + mid + high-level	Adds KNN/SVM or another classifier to interpret extracted features.	Grade label and optional confidence

2. Implementation Examples

```
# Low-level: smooth and threshold
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)
_, mask = cv2.threshold(blur, 90, 255, cv2.THRESH_BINARY_INV)
```

```
# Mid-level: extract connected regions and measurements
contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
                               cv2.CHAIN_APPROX_SIMPLE)
features = []
for c in contours:
    area = cv2.contourArea(c)
    x, y, w, h = cv2.boundingRect(c)
    features.append([area, w, h])

# High-level: use a trained classifier on validated features
# prediction = classifier.predict(feature_vector)
```

3. Performance Comparison

Parameter	**A. Low-level**	**B. Low + Mid**	**C. Low + Mid + High**
Ordinary-fruit accuracy (reported)	53.3%	91.1%	88.9%
Processing time	Fast	Fast/moderate	Model-dependent
Touching fruit	Poor	Depends on segmentation	Can learn patterns, still challenging
Leaf occlusion	Poor	Partly handled	Training-data dependent
Illumination robustness	Low	Improved with features	Depends on training and preprocessing
Explainability	High for simple rules	High	Moderate
New grade addition	Threshold change	Feature/rule update	Requires labelled examples/retraining

The reference summary reports the accuracy values above for its test set. Its reported timing was below the 1.2-second decision limit. The exact timing should be measured on the target edge computer before deployment.

4. Selected Pipeline Diagram

Level	Processing stages	Input	Output
Low-level	Denoising, HSV conversion, thresholding	Raw BGR image	Processed image and mask
Mid-level	Segmentation, contours, dimensions, colour and defect features	Mask and image	Regions and feature vector
High-level	KNN/SVM classification and buyer-rule decision	Feature vector	Export / reject / review

The selected pipeline is low + mid + high-level vision because it retains interpretable image operations while allowing a classifier to combine multiple features. A confidence threshold can route uncertain cases to human review.

5. Consequences of Inadequate Processing

- Low-level-only processing may interpret a dark stem end or shadow as a black spot, increasing false rejection.
- Poor segmentation may merge two touching mangoes, producing incorrect dimensions and defect counts.
- If mid-level features omit streak shape or local brightness variation, the high-level classifier cannot recover information that was never extracted.
- Occlusion by leaves requires better segmentation, controlled fruit presentation or additional camera views.

6. CNN and Vision Transformer Comparison

Model	Representation learning	Strength	Limitation
CNN	Learns local edges, textures and hierarchical features.	Effective for image classification and defect detection.	Needs labelled examples and careful validation.
Vision Transformer	Uses image patches and attention to model relationships.	Can capture long-range context.	May require more data and computation.
Traditional CV	Uses explicitly designed colour, shape and texture features.	Interpretable and lightweight.	Less flexible for complex appearance changes.

Pretrained models can reduce the amount of task-specific training data, but the required number depends on image variation, defect rarity and fine-tuning strategy. The reference discusses CNN/ViT data requirements but does not demonstrate a full trained model comparison.

Final Decision: A low + mid + high-level pipeline is selected. It provides measurable features and a final classification while allowing uncertain results to be reviewed. Segmentation quality remains the main limitation for touching fruits and severe occlusion.

TASK 3 – Digital Image Fundamentals I: Compare Image Representations for Ripeness [4 Marks]

A digital image is a function f(x,y) represented as an M × N array. Mango ripeness is related to peel colour, so RGB/BGR, HSV and CIE Lab representations are compared.

QUESTION AND REQUIREMENTS

A digital image is a function f(x, y) stored as an M × N (× channels) array of numbers. Ripeness depends on peel colour, so the way colour is represented matters. Compare three representations.

Apply (using Python/OpenCV): implement or simulate all three techniques on your own images and submit the commented code; every result below must come from running your own code or your own measurements.

Compare on: separation of green / turning / ripe classes, sensitivity to brightness change, effect of red blush, behaviour of hue near 0°/180° wrap-around, and computation time.

Justify with evidence: plot the class distributions (histograms or scatter plots) of the chosen channels for at least 30 fruits under two lighting levels, and report classification accuracy for each representation.

Show the array view of your image: shape, dtype, the (row, column) vs (x, y) convention in NumPy, the storage formula b = M × N × k bits, and the uint8 overflow difference between cv2.add and NumPy "+".

What if not: What if ripeness is graded in RGB and the roof daylight doubles? What if the image is accidentally treated as RGB instead of BGR?

Compare your chosen colour space with features from a pretrained network (for example, a ResNet or DINOv2 embedding) for ripeness on the same 30 fruits, and judge which is easier to explain to the buyer.

Evidence Expected: Channel plots and an accuracy table for the three representations, a justified choice, and a correct array/storage calculation.

Technique	**Basic idea and key concepts / functions**
A. RGB (BGR in OpenCV)	Thresholds on the three raw channels read by cv2.imread (BGR order).

B. HSV	cv2.cvtColor(img, cv2.COLOR_BGR2HSV); ripeness from hue H, with S and V used to ignore background and highlights.
C. CIE Lab*	cv2.cvtColor(img, cv2.COLOR_BGR2LAB); ripeness from a and b; colour difference ΔE between fruit and reference colours.

Ans:

1. Three Colour Representations

Representation	**Description**	**Useful channels**
RGB/BGR	Stores red, green and blue intensities; OpenCV reads BGR by default.	B, G, R
HSV	Separates colour type from saturation and brightness.	H, S, V
CIE Lab	Separates lightness from green–red and blue–yellow colour axes.	L*, a*, b*

2. Python Implementation

```
img = cv2.imread("mango.jpg")  # OpenCV loads BGR
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lab = cv2.cvtColor(img, cv2.COLOR_BGR2LAB)
b, g, r = cv2.split(img)
h, s, v = cv2.split(hsv)
l, a, bb = cv2.split(lab)
```

3. Comparison Under Illumination Changes

Criterion	RGB/BGR	HSV	CIE Lab
Green/turning/ripe separation	Moderate	Good using hue	Good using a* and b*
Brightness sensitivity	High	Hue is less sensitive; saturation/value still change	Lightness is separated but not fully invariant
Red blush	Changes channel values	Can be excluded with hue mask	Can be modelled using a*
Hue wrap-around	Not applicable	Must use circular hue handling	Not applicable
Computation	Low	Moderate	Moderate

The reference reports 100% HSV accuracy under the tested normal, dim and bright lighting conditions. It reports RGB accuracy of 53.3% in bright light. Lab performance was reported as generally high, but the exact full per-condition values should be taken from the executed experiment rather than inferred.

4. Array Shape, Data Type and Storage

```
print(img.shape)  # e.g. (480, 640, 3)
print(img.dtype)  # uint8
pixel = img[100, 200]  # row, column = y, x
```

For an image of 640 × 480 pixels with 3 channels and 8 bits per channel: storage = 640 × 480 × 24 = 7,372,800 bits = 921,600 bytes (approximately 0.92 MB), before compression.

5. uint8 Overflow: cv2.add versus NumPy +

```
a = np.array([250], dtype=np.uint8)
b = np.array([20], dtype=np.uint8)
print(cv2.add(a, b))  # [255], saturated addition
```

```
print(a + b)        # [14], uint8 wrap-around
```

If RGB thresholds are used while daylight intensity changes, the same fruit may cross a fixed threshold and be assigned to a different ripeness class. If BGR data is mistakenly interpreted as RGB, red and blue are interchanged, producing incorrect colour interpretation.

6. Pretrained Feature Comparison

A pretrained ResNet or DINOv2 model can convert an image into an embedding that represents colour, texture and shape. Such features may help with complex appearance variation, but they are less directly explainable to a buyer than a calibrated hue or Lab measurement. The reference discusses this comparison but does not report an executed pretrained-feature experiment.

Final Decision: HSV is selected because hue provides a practical separation of green, yellow and orange peel colours and achieved 100% in the reference lighting experiment. Circular hue averaging and blush masking are needed around the red hue boundary.

TASK 4 – Digital Image Fundamentals II: Compare Pixel Connectivity for Defect Counting [4 Marks]

After thresholding, connected-component labelling groups foreground pixels into individual defects. The buyer rejects more than three black spots or any spot larger than 4 mm. Four-connectivity, eight-connectivity and mixed connectivity are compared.

QUESTION AND REQUIREMENTS

The buyer rejects a fruit with more than 3 black spots or any spot larger than 4 mm. After thresholding, how pixels are grouped into "spots" decides the count. Compare three connectivity rules.

Develop three connected-component labelling techniques using Python/OpenCV to identify and count fruit defects. Submit commented source code and generate experimental results through code execution or actual measurements.

Examine the behaviour of the three techniques with respect to touching and diagonally adjacent spots, double-path ambiguity, noise-pixel sensitivity, variations in measured spot size, and execution time.

Validate the defect-counting performance using at least five synthetic binary patterns and ten thresholded fruit images. Present a tabular comparison of automatically detected spot counts and manually verified reference counts, supported by experimental evidence.

Calculate defect diameters and inter-spot gaps using Euclidean (De), city-block (D4), and chessboard (D8) distance measures. Convert pixel measurements into millimetres using image calibration and establish which measurement method satisfies the buyer's 4 mm criterion.

Investigate the effects of 4-connectivity splitting a diagonal sap-burn streak into multiple components and 8-connectivity combining separate defects into a single component. Formulate an appropriate connectivity-selection strategy for reliable fruit inspection.

Recommend a suitable defect-segmentation approach by comparing connectivity-based labelling with instance-segmentation models, such as YOLO-seg or SAM 2, using the same test images. Support the recommendation with evidence on individual defect masks, spot counts, touching and diagonal defects, measurement accuracy, and computational cost.

Evidence Expected: Labelled output images, a count comparison table, a distance-measure table, and a justified connectivity choice.

Technique	Basic idea and key concepts / functions
A. 4-connectivity	Pixels joined only through N4 neighbours; cv2.connectedComponentsWithStats(binary, connectivity=4).
B. 8-connectivity	Pixels joined through N8 neighbours (including diagonals); cv2.connectedComponentsWithStats(binary, connectivity=8).
C. m-connectivity (mixed)	Diagonal links allowed only when no shared 4-neighbour exists; implement your own labelling in Python (not built into OpenCV).

Ans:

1. Connectivity Techniques

Technique	Neighbour rule	Behaviour
4-connectivity	Top, bottom, left and right neighbours.	Diagonal pixels remain separate.
8-connectivity	All eight surrounding neighbours.	Diagonal pixels are joined.
m-connectivity	4-neighbours plus diagonal links only when common 4-neighbour conditions avoid ambiguous paths.	Reduces connectivity ambiguity.

2. Python Implementation

```
# 4-connected and 8-connected components
num4, labels4, stats4, cent4 = cv2.connectedComponentsWithStats(
    binary, connectivity=4)
num8, labels8, stats8, cent8 = cv2.connectedComponentsWithStats(
```

```
    binary, connectivity=8)
print("4-connected objects:", num4 - 1)
print("8-connected objects:", num8 - 1)
```

A full m-connectivity implementation uses a flood-fill or union-find procedure and checks the common 4-neighbours before accepting a diagonal connection. It should not be replaced by ordinary 4-connectivity alone.

3. Distance Measures

Measure	Formula	Meaning
Euclidean (De)	sqrt((x2−x1)^2 + (y2−y1)^2)	Straight-line distance
City-block (D4)	|x2−x1| + |y2−y1|	Horizontal/vertical path
Chessboard (D8)	max(|x2−x1|, |y2−y1|)	Allows diagonal steps

Pixel distance is converted into millimetres using: physical distance = pixel distance × mm per pixel. For example, at 0.3125 mm/pixel, a measured 10-pixel diameter equals 3.125 mm and is below the 4 mm limit.

4. Experimental Comparison and Segmentation

Criterion	4-connectivity	8-connectivity	m-connectivity
Diagonal streak	May split into several regions	Remains connected	Conditional diagonal joining
Touching spots	May remain separate	May merge	Depends on local pattern
Noise sensitivity	Small isolated pixels may count	Diagonal noise may join regions	Still needs area filtering
Execution	Fast	Very fast in OpenCV	Custom implementation is slower

The reference experiment reports that 4-connectivity divided a diagonal sap-burn streak into nine components. It also reports that the tested m-connectivity patterns produced the same component count as 8-connectivity, while custom m-connectivity took longer. These are test-specific results, not universal guarantees.

YOLO-seg and SAM 2 can produce object masks and may separate touching defects when the model is suitable. They require model resources and validation. The reference discusses them but does not report an executed comparison.

Final Decision: 8-connectivity with a minimum-area filter and Euclidean distance is selected for the fast production pipeline. The strategy should be validated on diagonal streaks and touching defects; uncertain merged regions can be sent for review.

TASK 5 – Image Sensing and Acquisition: Compare Sensor Arrangements [5 Marks]

Mangoes move at 0.5 m/s beneath the camera. The acquisition arrangement must produce sharp, geometrically reliable and correctly illuminated images. Area-scan, rolling/global-shutter and line-scan arrangements are compared.

QUESTION AND REQUIREMENTS

The fruit moves at 0.5 m/s under the camera. How the scene energy is sensed decides whether the image is sharp, undistorted and correctly coloured. Compare three acquisition arrangements.

Demonstrate three imaging arrangements using Python/OpenCV: (A) area-scan imaging with motion blur, (B) rolling-shutter imaging with row-wise displacement, and (C) line-scan imaging through sequential line acquisition and stacking. Where feasible, capture images of a moving object and submit commented source code with experimentally obtained results.

Examine the three imaging arrangements with respect to motion blur, calculated as belt speed multiplied by exposure time; rolling-shutter skew; sensitivity to 100 Hz illumination flicker; colour acquisition using a Bayer colour filter array versus a three-sensor camera; required frame or line rate; and implementation cost.

Compute motion blur for exposure times of 0.1 ms, 1 ms, and 4 ms using the specified belt speed. Determine the required line-scan rate for Arrangement C and experimentally verify whether a 2 mm defect remains detectable under each simulated imaging condition.

Justify the selection of CCD or CMOS sensor technology for the proposed inspection system. Illustrate the signal-conversion sequence from incident photons to electrical charge or voltage, analogue signal conditioning, analogue-to-digital conversion, and the resulting digital pixel value.

Predict the effects of using Arrangement A with a 4 ms exposure and operating Arrangement C without belt synchronisation when belt speed varies by 10%. Recommend suitable corrective measures to preserve image quality and dimensional accuracy.

Assess the suitability of event cameras and multispectral or hyperspectral cameras for high-speed fruit sorting. Explain their operating principles and potential inspection applications, and justify whether their temporal resolution, spectral information, integration requirements, and cost are appropriate for the proposed production line.

Evidence Expected: Blur/skew calculations, simulated or captured images for all three arrangements, a comparison table, and a justified sensor choice.

Technique	Basic idea and key concepts / functions
A. Area-scan rolling-shutter CMOS (USB)	Sensor array with rows exposed one after another; continuous lighting; cv2.VideoCapture with manual exposure (CAP_PROP_EXPOSURE).
B. Area-scan global-shutter CMOS with strobe	Sensor array with all pixels exposed together; short LED flash triggered by a belt encoder.
C. Line-scan camera (sensor strip)	A single row of sensors; the belt motion builds the 2-D image line by line, synchronized to an encoder.

Ans:

1. Three Acquisition Arrangements

Arrangement	Principle	Main issue / benefit
A. Area-scan rolling shutter	Rows are exposed/read at different times.	Can show skew and motion distortion.
B. Area-scan global shutter + strobe	All pixels are exposed together during a short flash.	Reduces blur and row-wise skew.
C. Line scan	One sensor line is captured repeatedly and stacked as the conveyor moves.	Needs accurate encoder synchronisation.

2. Motion Blur Calculation

Motion blur is $B = v \times t$. At 0.5 m/s, $v = 500$ mm/s.

Exposure	Calculation	Blur
0.1 ms	500×0.0001	0.05 mm
1 ms	500×0.001	0.5 mm

4 ms	500 × 0.004	2.0 mm

A 4 ms exposure blurs the image by 2 mm, which is equal to the smallest original defect. A short strobe exposure is therefore preferred.

3. Rolling Shutter, Global Shutter and Line Scan

Criterion	Rolling shutter	Global shutter + strobe	Line scan
Motion distortion	Rows may shift	No rolling row skew	Depends on encoder synchronisation
Blur	Depends on exposure	Very low with short strobe	Low if line rate matches belt
Lighting flicker	Can produce bands	Short flash reduces effect	Requires controlled line light
Installation	Simple	Moderate	More complex
Cost	Low	Medium	Higher integration cost

At a line spacing of 0.3125 mm and a belt speed of 500 mm/s, the required line rate is 500 / 0.3125 = 1,600 lines/s.

4. Sensor and Signal Chain

CCD sensors transfer charge across the sensor before readout. CMOS sensors convert pixel charge to voltage using integrated pixel circuitry and provide fast readout with low power. A global-shutter CMOS camera is selected for the original production line.

Signal-chain stage	Function
Photodiode	Converts incident photons into electrical charge.
Charge-to-voltage conversion	Converts accumulated charge into voltage.
Analogue conditioning	Amplifies and conditions the signal.

ADC	Converts the analogue signal into digital values.
Digital output	Provides pixel values to the processing computer.

5. Flicker and Advanced Camera Types

A 50 Hz tube light may produce 100 Hz intensity variation, causing bright/dark bands. A synchronised LED strobe and controlled enclosure reduce the effect. Event cameras detect changes in brightness asynchronously; multispectral cameras capture selected wavelength bands such as RGB and NIR; hyperspectral cameras capture many narrow bands but are expensive and data intensive.

Final Decision: Global-shutter CMOS with a short LED strobe is selected. The reference simulation reports approximately 0.05 mm blur and no rolling-shutter skew for the selected short exposure. Real moving-belt validation is still required.

TASK 6 – Sampling: Compare Spatial Resolutions [5 Marks]

Spatial sampling determines how many pixels represent the 400 mm belt width. It affects the number of pixels across a 2 mm defect, dimensional accuracy, processing time and storage.

QUESTION AND REQUIREMENTS

Sampling decides how many pixels cover the 400 mm field of view, and therefore the smallest detail that can be seen. Compare three camera resolutions.

Implement image down-sampling in Python/OpenCV using cv2.resize with INTER_NEAREST and INTER_AREA at the specified resolutions. Submit commented source code and obtain all reported results through code execution or actual measurements.

Analyze the effect of image resolution on millimetres per pixel, the number of pixels spanning a 2 mm defect, dimensional measurement error, aliasing, edge distortion, data rate at the required frame rate, seven-day storage requirements, and processing time.

Verify the suitability of each resolution using the Nyquist sampling principle, requiring at least two samples across the smallest defect and preferably three to four pixels for practical detection. Measure defect-detection rate and size-estimation error on the re-sampled images and substantiate the selected resolution with experimental evidence.

Determine the minimum frame rate required to capture each fruit completely in at least one frame, given a 300 mm field of view along the conveyor, a maximum fruit length of 110 mm, and a belt speed of 0.5 m/s.

Predict the consequences of selecting the lowest-resolution arrangement to reduce cost by quantifying missed 2 mm defects and assessing compliance with the 2% defect-miss limit. Examine whether the edge-processing computer can sustain the required throughput when the highest-resolution arrangement is selected.

Critique the effectiveness of deep-learning super-resolution techniques, such as Real-ESRGAN, in recovering the visibility of missed 2 mm defects from low-resolution images. Distinguish genuine defect evidence from generated image detail and assess the implications for measurement reliability and buyer-audit compliance.

Evidence Expected: A resolution table (mm/pixel, pixels per defect, data rate, storage, time), down-sampled image comparisons, and a justified resolution.

Technique	Basic idea and key concepts / functions

A. 640 × 480 (0.3 MP)	Low resolution; small images and fast processing.
B. 1280 × 1024 (1.3 MP)	Medium resolution; a common industrial choice.
C. 2448 × 2048 (5 MP)	High resolution; large images and slower processing.

Ans:

1. Three Spatial Resolutions

Resolution	Approx. megapixels	mm/pixel across 400 mm	Pixels across 2 mm
640 × 480	0.3 MP	400/640 = 0.625	3.2
1280 × 1024	1.3 MP	400/1280 = 0.3125	6.4
2448 × 2048	5 MP	400/2448 = 0.1634	12.24

The Nyquist principle requires at least two samples across the smallest feature; in practical inspection, three to four or more pixels are preferred. The 1.3 MP and 5 MP options provide more margin than the lowest-resolution option.

2. Down-Sampling Implementation

```
low = cv2.resize(img, (640, 480), interpolation=cv2.INTER_NEAREST)
medium = cv2.resize(img, (1280, 1024), interpolation=cv2.INTER_AREA)
high = cv2.resize(img, (2448, 2048), interpolation=cv2.INTER_AREA)
```

INTER_NEAREST selects the nearest source pixel. INTER_AREA uses area-based resampling and is generally suitable when reducing image size. Down-sampling must be evaluated with the same defect detector and ground truth.

3. Frame Rate Requirement

The field of view along the conveyor is 300 mm and maximum fruit length is 110 mm. At 500 mm/s, the available travel distance is 300 − 110 = 190 mm. The theoretical minimum capture

opportunity rate is 500 / 190 = 2.63 frames/s. The actual production trigger rate must also satisfy the 8-fruit/s throughput and fruit spacing requirements.

4. Storage and Processing Comparison

Criterion	640 × 480	1280 × 1024	2448 × 2048
Spatial detail	Lowest	Moderate/good	Highest
2 mm defect coverage	3.2 px	6.4 px	12.24 px
Processing load	Lowest	Moderate	Highest
Data volume	Lowest	Moderate	Highest
Suitability	Marginal for faint defects	Balanced	Useful if edge PC sustains throughput

For an uncompressed 8-bit RGB image, storage bytes = M × N × 3. Daily storage = bytes per frame × frame rate × 86,400. Seven-day storage = daily storage × 7. Actual archive size depends on image crop, compression, frame selection and metadata.

5. Super-Resolution Limitation

Real-ESRGAN can enlarge an image and generate plausible detail, but it cannot prove that a defect existed in the original low-resolution capture. Generated texture should not be treated as genuine measurement evidence for buyer audits.

Final Decision: The 1280 × 1024 (1.3 MP) arrangement is selected for the original line because it provides 6.4 pixels across a 2 mm defect and balances image detail, storage and processing load. It must be confirmed on the actual four-core edge PC.

TASK 7 – Quantization: Compare Intensity Resolutions [4 Marks]

Quantization maps image intensity into a finite number of levels. A bruise may be only 6–10 grey levels darker than healthy peel, so aggressive quantization can remove the contrast needed for detection.

QUESTION AND REQUIREMENTS

A bruise is only 6–10 grey levels (8-bit) darker than healthy peel. Quantization decides whether this small difference survives. Compare three intensity resolutions.

Implement three image-quantization techniques using Python/OpenCV on fruit images and submit commented source code. Obtain all reported results through actual code execution or experimental measurements.

Analyze the effects of quantization on bruise visibility across intensity levels, false contouring on smooth fruit peel, colour difference (ΔE), image file size, and seven-day storage requirements.

Verify the effects of quantizing the same fruit images to 8, 6, 5, and 4 bits by measuring bruise contrast and detection rate, calculating daily storage requirements, and presenting visual evidence of false contouring.

Determine the optimal trade-off between spatial sampling and intensity quantization under a fixed storage budget. Justify whether retaining more spatial pixels or more intensity levels provides greater benefit for detecting bruises on the production line.

Assess whether fruit images stored at 5-bit depth on a 2 TB storage device retain sufficient visual evidence to investigate buyer complaints involving bruised fruit. Support the assessment with measured contrast, detection performance, and storage calculations.

Differentiate image bit depth from deep-learning model precision by examining HDR image acquisition and INT8 model quantization for edge deployment. Explain their respective effects on bruise visibility, defect-detection accuracy, and computational requirements.

Evidence Expected: Re-quantized image series, a table of contrast, detection rate and storage per bit-depth, and a justified bit-depth.

Technique	Basic idea and key concepts / functions
A. 8-bit (256 levels)	Standard image depth; uint8 arrays in OpenCV.

B. 12-bit raw (4096 levels)	Camera raw output stored as uint16; scaled for display (cv2.normalize / convertScaleAbs).
C. Reduced 4–6 bit (16–64 levels)	Re-quantize to save storage: img // step * step (or cv2.LUT).

Ans:

1. Quantization Techniques

Technique	Number of levels	Description
8-bit	256	Standard 8-bit image values from 0 to 255.
12-bit raw	4096	Higher precision acquisition, commonly stored in uint16.
Reduced 4–6 bit	16–64	Fewer levels to reduce storage, with risk of lost contrast.

2. Re-quantization Code

```
def requantize(img8, bits):
    step = 2 ** (8 - bits)
    q = (img8 // step) * step
    # Mid-step reconstruction for reduced bit depths
    if bits < 8:
        q = q + step // 2
    return np.clip(q, 0, 255).astype(np.uint8)

img8 = cv2.imread("mango.jpg", cv2.IMREAD_GRAYSCALE)
img6 = requantize(img8, 6)
img5 = requantize(img8, 5)
```

```
img4 = requantize(img8, 4)
```

3. Intensity Resolution and Bruise Visibility

Bit depth	Levels	Step in 8-bit scale	Expected effect
8-bit	256	1	Preserves small differences
6-bit	64	4	May retain some faint bruise contrast
5-bit	32	8	A 6–10 level bruise may be weakened or lost
4-bit	16	16	Small bruise contrast may disappear
12-bit	4096	Finer raw precision	Useful if sensor noise and pipeline support the extra precision

The reference experiment reports 100% bruise detection at 8 bits, 91.7% at 5 bits and 83.3% at 4 bits. The exact 6-bit result must be read from the actual experiment output; it is not inferred here.

4. Storage and Seven-Day Retention

For the same image dimensions, uncompressed storage is proportional to bits per pixel. The reference reports approximately 110 GB/day for its 8-bit archive and 46 GB/day for its 5-bit archive; the seven-day 8-bit crop archive is approximately 770 GB, within the 2 TB capacity. These figures depend on the reference image crop and capture policy.

5. Image Bit Depth versus INT8 Model Precision

Aspect	HDR / high-bit-depth acquisition	INT8 model quantization

Changes	Image acquisition precision and dynamic range.	Neural-network weights and activations.
Purpose	Preserve scene information in dark and bright regions.	Reduce memory and computation.
Effect on bruise	Can preserve faint intensity differences.	Cannot restore image contrast already lost.
Deployment	May require more storage/bandwidth.	May accelerate inference on supported hardware.

At 5-bit storage, the reference reports false contour bands and reduced bruise detection. This makes 5-bit images unsuitable as the only evidence for buyer complaints even if storage is reduced.

Final Decision: 8-bit lossless storage is selected. It preserves the tested bruise contrast, avoids the false contours reported at 4/5 bits and fits the seven-day crop archive within 2 TB. HDR may be considered at acquisition where highlights and shadows cause clipping.

TASK 8 – Image Formation Models: Compare Optics and Lighting Configurations [5 Marks]

Image formation has a geometric component, which determines object projection and apparent size, and a photometric component, which determines pixel brightness. Three optics and lighting arrangements are compared.

QUESTION AND REQUIREMENTS

5 Marks

Two models explain the image: the geometric model (pinhole/perspective projection $x = f \cdot X/Z$ decides where and how large the fruit appears) and the photometric model $f(x, y) = i(x, y) \cdot r(x, y)$ (illumination × reflectance decides how bright each pixel is). Compare three imaging configurations.

Emulate three imaging configurations using Python/OpenCV, employing a cardboard enclosure lined with white paper as a dome and objects of known dimensions placed at two different heights. Submit commented source code and obtain all reported results through actual measurements or code execution.

Analyze the three configurations with respect to dimensional measurement error between the top of a 90 mm-high fruit and the conveyor plane, lens distortion, illumination uniformity (i(x,y)), specular reflections from waxed fruit surfaces, stability of reflectance-based thresholds (r(x,y)) throughout the day, and overall cost.

Calculate the focal length required to achieve a 400 mm field of view at a working distance of 500 mm using the selected sensor width. Quantify the perspective-induced size error for a 90 mm-high fruit and measure illumination non-uniformity and changes in the optimal defect-detection threshold between morning and noon images.

Demonstrate the effectiveness of a processing technique that separates illumination and surface reflectance, using either white-reference flat-field correction or homomorphic filtering based on (\log f=\log i+\log r). Present the resulting images and compare defect-detection performance before and after correction.

Predict the consequences of ignoring perspective effects on fruit size classification and quantify the resulting misclassification rate. Examine the impact of ambient-light variation, such as cloud

cover over the inspection area, and propose suitable illumination-control and calibration measures.

Assess the suitability of monocular depth-estimation networks, such as Depth Anything, for estimating fruit height by comparing their measurements with calibrated geometric measurements and telecentric imaging. Determine whether each method can meet the required dimensional accuracy of ±2 mm, supported by experimental evidence.

Evidence Expected: Projection and focal-length calculations, measured size errors and illumination maps for the three configurations, and a justified optics–lighting choice.

Technique	Basic idea and key concepts / functions
A. Ambient roof light + standard lens	Perspective projection with a fixed-focal-length lens; illumination i(x, y) from daylight and tube lights.
B. Diffuse LED dome + standard lens + calibration	Controlled, even illumination; camera calibrated with a checkerboard (cv2.findChessboardCorners, cv2.calibrateCamera) and height-corrected mm/pixel.
C. Diffuse LED dome + telecentric lens	Near-orthographic projection – object size does not change with distance from the lens.

Ans:

1. Geometric and Photometric Models

Geometric projection: $x = fX/Z$. Photometric model: $f(x,y) = i(x,y) \times r(x,y)$, where i is illumination and r is surface reflectance.

Configuration	Description	Main advantage / limitation
A. Ambient roof light + standard lens	Uses daylight and tube lights with a standard perspective lens.	Low cost; illumination and perspective vary.

B. Diffuse LED dome + standard lens + calibration	Controlled lighting, checkerboard calibration and height correction.	Good balance of uniformity, accuracy and cost.
C. Diffuse LED dome + telecentric lens	Near-orthographic imaging with low magnification change over depth.	Excellent dimensional stability; high cost.

2. Focal Length Calculation

Using the thin-lens field-of-view approximation: f = (sensor width × working distance) / FOV. For a 6.4 mm sensor width, 500 mm working distance and 400 mm field of view: f = (6.4 × 500) / 400 = 8 mm.

3. Perspective Error for a 90 mm Fruit

Camera-to-conveyor distance = 500 mm. Camera-to-top-of-fruit distance = 500 − 90 = 410 mm. The apparent scale ratio is 500/410 = 1.2195. Thus, the top of the fruit can appear approximately 21.95% larger than the same object at conveyor level if height/perspective is ignored.

4. Illumination Correction

White-reference flat-field correction estimates spatial illumination variation. A simple form is corrected image = original image / (reference image + ε) × scale. Homomorphic filtering uses log f = log i + log r to separate slowly varying illumination from reflectance detail.

```
img = cv2.imread("mango.jpg").astype(np.float32)
ref = cv2.imread("white_reference.jpg").astype(np.float32)
corrected = img / (ref + 1e-6)
corrected = cv2.normalize(corrected, None, 0, 255,
                          cv2.NORM_MINMAX).astype(np.uint8)
```

The reference reports illumination non-uniformity decreasing from 62% to 1% after correction, and false defect flags decreasing from 1.15% to 0% in its tested setup. The dome was emulated in software rather than physically constructed.

5. What-If Analysis and Depth Estimation

- Ignoring perspective can cause incorrect size classification because fruit height changes magnification.
- Cloud cover and daylight changes alter image brightness and can shift fixed defect thresholds.
- An enclosed hood, diffuse LED lighting, white-reference correction and periodic calibration improve stability.

Monocular depth networks such as Depth Anything estimate relative or learned depth from a single image. Their output must be calibrated and validated against geometric measurements. The reference did not run Depth Anything, so compliance with ±2 mm is not experimentally established.

Final Decision: A diffuse LED dome with a calibrated standard lens is selected for the original system. It offers controlled illumination and corrected dimensional measurements at lower cost than a telecentric lens. A telecentric lens is appropriate if tighter dimensional tolerance justifies its cost.

TASK 9 – Applications and Integrated System: Compare Complete Front-End Configurations [6 Marks]

The decisions from Tasks 1–8 are combined into a complete front-end for mango inspection. The system must meet 8 fruits/s, a 1.2-second decision limit, seven-day retention and a ₹1,50,000 imaging budget.

QUESTION AND REQUIREMENTS

Combine your choices from Tasks 1–8 into one imaging front-end. Compare three complete configurations within the ₹1,50,000 budget (use real vendor prices and cite them).

Develop and execute a fruit-grading prototype using Python/OpenCV on the same labelled image dataset prepared for the three imaging configurations. Submit commented source code and obtain all experimental results through actual code execution or measurements.

Examine the performance of the three configurations with respect to grading accuracy against the 2% defect-miss limit, decision latency against the 1.2-second limit, implementation cost, seven-day storage requirements, robustness to illumination changes, motion, and fruit-height variations, and maintenance effort.

Assess the experimental results against the predictions established in Tasks 1–8. Present a comparative results table containing measured grading performance and processing time, including challenging cases such as touching fruits, glossy surface reflections, and cloud-induced shadows. Identify which predictions are supported or contradicted by the evidence.

Design an end-to-end system block diagram covering scene, illumination, optics and image formation, sensor, sampling and quantization, digital image, low-level processing, mid-level processing, high-level processing, grading decision, and ejector. Allocate a time budget to each processing stage and construct a Design Decision Table containing Decision, Justification (Evidence), and What If Not.

Differentiate three additional applications of image processing and computer vision, such as medical X-ray analysis, satellite remote sensing, and vehicle number-plate recognition. Tabulate the changes in sensing band, spatial sampling, quantization, and image-formation model for each application relative to fruit inspection.

Investigate the limitations of testing a grading system exclusively on clean, well-illuminated images. Identify at least one experimental result that disagrees with theoretical expectations and

explain the discrepancy using measured evidence, possible sources of error, and the limitations of the experimental setup.

Recommend a suitable industrial inspection configuration by incorporating a fourth option: a smart camera with an onboard AI accelerator running a compact object detector. Compare all four configurations in terms of acquisition and processing latency, total system cost, seven-day storage, robustness, deployment complexity, and retraining or rule-maintenance effort.

Evidence Expected: A measured comparison of three configurations, an original block diagram and decision table, an applications transfer table, and a defended final design.

Technique	Basic idea and key concepts / functions
A. Budget configuration	USB rolling-shutter 0.3–1.3 MP camera, ambient light, 8-bit JPEG storage, low + mid-level pipeline.
B. Balanced configuration	Global-shutter 1.3 MP GigE camera, LED dome with strobe, calibrated standard lens, 8-bit lossless storage, three-level pipeline.
C. Premium configuration	High-resolution line-scan or 5 MP camera, telecentric lens, 12-bit storage, three-level pipeline.

Ans:

1. Three Configurations

Configuration	Main components	Processing approach
A. Budget	Low-cost rolling-shutter USB camera, ambient lighting and compressed images.	Basic pixel and feature rules.
B. Balanced	Global-shutter camera, LED dome/strobe, calibrated standard lens and lossless 8-bit crop storage.	Low/mid/high-level vision with rule-based decision.

C. Premium	Higher-resolution camera, telecentric optics and higher bit-depth acquisition.	More demanding processing and calibration.

2. Reported Performance Comparison

Parameter	Budget	Balanced	Premium
Accuracy (reference test)	50%	93.8%	87.5%
Processing time (reported)	~35 ms	31 ms	~140 ms or more
False rejection	All good fruit in tested case	0% in tested case	Some false rejection
Illumination robustness	Poor	Good	Good
Cost	Lowest	Within ₹1.5 lakh	Over budget

These are the reference document's test results and not independently verified vendor or production-line specifications. The Premium configuration did not outperform Balanced in the reported test; the reference attributes this partly to an algorithm that was not retuned for the higher resolution.

3. Integrated Architecture

Stage	Function	Output
Illumination	Diffuse LED dome and short strobe.	Stable illuminated scene
Optics and sensor	Calibrated lens and global-shutter CMOS.	Sharp image
Sampling/quantization	1280 × 1024, 8-bit lossless crop.	Digital image

Low-level processing	Denoise, HSV conversion and segmentation.	Mask
Mid-level processing	Contours, dimensions, hue and defect features.	Feature vector
High-level decision	Apply buyer rules and classifier/review threshold.	Export / reject / review
Control	Communicate result to ejector.	Physical sorting

4. Processing Time Budget

Stage	Illustrative allocation
Acquisition	5 ms
Preprocessing	5 ms
Segmentation	5 ms
Feature extraction	6 ms
Classification	5 ms
Decision/communication	5 ms
Total	31 ms

The stage allocations are an illustrative decomposition of the reported 31 ms total; they are not separately measured timings. Actual end-to-end latency must be measured on the target edge PC, including camera transfer, queueing and ejector communication.

5. Design Decision and Transfer to Other Applications

Design choice	Reason	Consequence if omitted
Global-shutter + strobe	Limits motion blur and skew.	Faint defects may be missed.
1.3 MP sampling	6.4 pixels across 2 mm.	Small details may be lost.
8-bit lossless storage	Preserves tested bruise contrast.	False contours and audit evidence loss.

LED dome + flat-field	Stabilises illumination.	Thresholds drift with daylight.
Calibrated standard lens	Corrects measurement scale.	Perspective errors remain.
HSV hue	Stable ripeness cue in tested lighting.	RGB thresholds vary with brightness.
8-connectivity + area filter	Counts diagonal regions efficiently.	Streaks may be split or noise counted.

Application	**Sensing band**	**Spatial sampling**	**Quantization**	**Image formation**
Mango inspection	Visible RGB; RGB + NIR for revised requirement	~0.3 mm/px original design	8-bit	Perspective projection and reflected light
Medical X-ray	X-rays	~0.1–0.2 mm/px detector example	12–16 bit	Transmission / attenuation
Satellite remote sensing	Visible, NIR, SWIR, thermal	~0.3–30 m/px	10–16 bit	Push-broom scanning and atmospheric effects
Number-plate recognition	Visible + IR illumination	Around 1 mm/px on plate	8-bit	Perspective imaging and retroreflection

6. Limitations of Clean-Image Testing

A clean image set may omit touching fruit, glare, cloud changes, dust and lens contamination. Testing should include these difficult conditions. Synthetic images provide controlled ground truth but do not reproduce all natural fruit textures and optical effects. A pilot run on real conveyor images is required before deployment.

7. Smart Camera Option

A smart camera with an onboard AI accelerator may perform inference close to the sensor and reduce communication latency. However, it requires representative labelled examples, model validation, version control and maintenance. The reference estimates ₹1–2.5 lakh or more and recommends considering it after collecting several thousand labelled images.

Final Decision: The Balanced configuration is selected for the original requirement because it achieved 93.8% accuracy in the reference test, reported 31 ms processing time and remained within budget. It should be validated with real fruit, full shift conditions and the target edge PC.

TASK 10 – Re-apply and Adapt the Imaging Front-End [8 Marks]

A new export buyer requires fruit-fly sting and internal bruise detection down to 0.5 mm. Some bruises are visible only in near-infrared. The conveyor speed doubles to 1.0 m/s, throughput increases to 16 fruits/s, daylight variation increases and the decision must be made within 0.6 seconds. The existing budget and edge PC remain unchanged.

QUESTION AND REQUIREMENTS

A new export buyer in Japan requires detection of fruit-fly stings and internal bruises as small as 0.5 mm, some of which are invisible in normal colour images but visible in near-infrared (NIR). To meet demand, the belt speed doubles to 1.0 m/s (16 fruits per second), the shed roof gets skylights (even stronger daylight changes), and the same edge PC and the same ₹1,50,000 budget must be used. Decisions are still needed before the ejector, now only 0.6 s away.

Evaluate the impact of the revised inspection requirements on the existing system. Identify the original assumptions concerning defect size, conveyor speed, illumination, spectral band, decision time and data volume that are no longer valid, and justify each change. Assess the expected consequences of retaining the Task 9 front-end without modification.

Recalculate the spatial resolution (mm/pixel) required to detect a 0.5 mm defect, motion blur at a conveyor speed of 1.0 m/s, the required frame rate or line-scan rate, image bit depth and data acquisition rate. Evaluate the original front-end's performance against these requirements using the same computing platform. Identify the components and design assumptions that remain suitable and justify their continued use.

Compare Strategies A, B and C in terms of 0.5 mm defect detection, NIR-only defect detection, decision time, storage requirements, implementation cost and robustness. Use Python/OpenCV simulations, measured results and engineering calculations to substantiate the comparison. Recommend a revised image acquisition and processing front-end, justifying the selected strategy and the modifications required.

Justify the final front-end design using calculated performance requirements, Task 9 baseline measurements, experimental results, available budget and computing hardware limitations. Evaluate the trade-offs associated with resolution, acquisition speed, spectral capability, processing time and storage. State one specific operating condition under which the proposed design would no longer be suitable and justify the limitation.

Evaluate how few-shot learning with a pretrained foundation model could reduce the number of labelled images required to detect newly introduced NIR-specific defects. Discuss the risks associated with limited training samples, domain shift, false acceptance or rejection, model traceability and reproducibility. Recommend measures to maintain reliable defect classification and provide auditable evidence for export-quality inspection.

Evidence Expected: Three adaptation strategies applied and compared with measured and calculated evidence, and a RETAIN / MODIFY / REDESIGN decision that follows logically from the changed constraints.

Technique	Basic idea and key concepts / functions
A. Retain + ROI processing	Keep the Task 9 camera and lighting; shorten exposure, crop to a region of interest and process only the fruit area.
B. Modify sampling and acquisition	Higher-resolution global-shutter camera, shorter strobe, enclosed hood to block daylight, 10/12-bit depth.
C. Redesign sensing (multispectral)	RGB + NIR line-scan (or two-camera) sensing inside an enclosure, with new image formation and calibration.

Ans:

1. Original Assumptions That Must Change

Parameter	Original	Revised requirement	Effect
Smallest defect	2 mm	0.5 mm	Higher spatial resolution required
Belt speed	0.5 m/s	1.0 m/s	Shorter exposure / higher line rate
Throughput	8 fruits/s	16 fruits/s	Higher sustained processing rate

Illumination	Variable daylight	Stronger daylight variation	Enclosure and controlled lighting
Spectral band	Visible RGB	RGB + NIR	NIR sensing required
Decision time	1.2 s	0.6 s	Tighter end-to-end latency

2. Revised Spatial Resolution

For at least three pixels across a 0.5 mm defect: mm/pixel = 0.5/3 = 0.167 mm/pixel. Across a 400 mm field, required horizontal sampling is 400/0.167 ≈ 2,395 pixels. A camera with approximately 2,448 horizontal pixels is suitable.

3. Revised Motion Blur and Line Rate

At 1.0 m/s = 1,000 mm/s, a 0.05 ms exposure gives blur = 1,000 × 0.00005 = 0.05 mm. For half-pixel blur at 0.167 mm/pixel, the maximum exposure is approximately 0.0835 ms. A 0.05 ms strobe is therefore a suitable target.

Required line rate = 1,000 / 0.167 ≈ 6,000 lines/s. The line-scan camera must be synchronised to the conveyor encoder.

4. Frame Opportunity and Data Rate

With a 300 mm conveyor-direction field and 110 mm maximum fruit length, theoretical minimum capture opportunity rate = 1,000/(300−110) = 5.3 fps. This does not replace the 16 fruit/s trigger requirement; the production design must be based on actual fruit spacing and encoder triggers.

Data stream	Calculation	Approximate rate
5 MP area camera, 16-bit	5 MB × 16 frames/s	80 MB/s
RGB + NIR line scan	2448 × 6000 × 4 bytes	59 MB/s

5. Three Adaptation Strategies

Strategy	Description	Main limitation

A. Retain + ROI	Keep original camera, shorten exposure and process only fruit ROI.	Insufficient pixels across 0.5 mm and no NIR-only sensing.
B. Modify sampling/acquisition	Use 5 MP global shutter, 0.05 ms strobe and enclosed hood.	Improves visible detail but cannot detect NIR-only bruises.
C. Redesign sensing	Use RGB + NIR line-scan camera or synchronised cameras with encoder.	Higher integration and calibration complexity.

6. Reference Experimental Comparison

Strategy	0.5 mm stings detected	NIR-only bruises detected	Time per fruit	Within 0.6 s
A. Retain + ROI	0%	0%	32 ms	Yes
B. Modify	100%	0%	140 ms	Yes
C. RGB + NIR redesign	100%	100%	146 ms	Yes

The reference results were produced using simulated fruit images. Strategy A fails because 0.5 mm defects cover too few pixels at the original sampling. Strategy B improves spatial resolution but visible RGB cannot reveal a defect that is present only in NIR. Strategy C detects both types in the reference simulation.

7. Recommended Front-End

Component	Recommended specification / role
Imaging enclosure	Blocks changing daylight and stabilises the scene.
Illumination	Diffuse visible LED and NIR line illumination, including an 850 nm band where validated.

Sensor	RGB + NIR line-scan camera or synchronised cameras.
Spatial sampling	Approximately 0.167 mm/pixel.
Line rate	Approximately 6,000 lines/s at 1 m/s for the selected line spacing.
Synchronisation	Belt encoder controls line acquisition.
Processing	ROI-based RGB and NIR feature extraction; retain the original image evidence.
Decision	Export / reject / review before the 0.6-second ejector deadline.

8. Few-Shot Learning and Limitations

Few-shot learning can adapt a pretrained model to new defect categories using a small labelled set. It may help identify fruit-fly stings and unusual bruise patterns, but its reliability must be tested on unseen fruit, new lighting and different lots. Model version, training data, confidence and original images should be retained for audit.

Line-scan imaging can distort fruit images if the fruit rolls or bounces because the image is assembled from successive lines. Mechanical stabilisation, encoder monitoring and rejection of badly distorted captures are therefore important.

Final Decision: Strategy C – RGB + NIR multispectral redesign is selected. It is the only tested strategy in the reference simulation that detects both 0.5 mm stings and NIR-only bruises, while its reported 146 ms per-fruit processing time is within the 0.6-second decision window. The indicative cost is approximately ₹1.3–1.5 lakh and must be confirmed with vendor quotations.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Algorithm Design and Grading Comparison	4
Task 2 – Pipeline Design and Operation	5
Task 3 – Image Representation Analysis	4
Task 4 – Connectivity Analysis	4
Task 5 – Image Acquisition Analysis	5
Task 6 – Sampling Analysis	5
Task 7 – Quantization Analysis	4
Task 8 – Image Formation Analysis	5
Task 9 – Implementation and Performance Evaluation	6
Task 10 – Front-End Adaptation Challenge	8
TOTAL	50